

# Recommendation Systems for Personalized Content Discovery

*Item-Based Collaborative Filtering · Singular Value Decomposition*

Dataset: Netflix Prize (Kaggle) · Models: Item-CF & SVD · Evaluation @ K = 10  
Users Evaluated: 500 · Relevance Threshold:  $\geq 3.5$  stars

| Item-CF RMSE         | SVD MAP@10         | SVD NDCG@10        |
|----------------------|--------------------|--------------------|
| 0.9422               | 0.013685           | 0.039113           |
| Best Rating Accuracy | Best Ranking (SVD) | Best Ranking (SVD) |

## 1. Problem Understanding

The modern streaming era is defined by an overwhelming volume of content — yet most viewers engage with only a small fraction of what is available. Recommendation systems bridge this gap, transforming raw user interaction history into personalised discovery. This project addresses a canonical challenge: given a highly sparse user–item rating matrix derived from the Netflix Prize Dataset, build, evaluate, and compare two recommendation paradigms capable of predicting unseen ratings and generating meaningful Top-K recommendations.

### 1.1 Dataset Overview

The Netflix Prize Dataset — available on Kaggle — is one of the most influential benchmarks in recommendation systems research. Originally released in 2006 as part of a global competition to improve Netflix's CineMatch algorithm by more than 10%, it has since become the canonical reference for collaborative filtering, matrix factorisation, latent factor models, and cold-start research.

- ▶ 100,480,507 total movie ratings across the full corpus
- ▶ 480,189 unique users and 17,770 unique movies
- ▶ Ratings on a 1–5 integer star scale with associated timestamps
- ▶ Each record includes: User ID, Movie ID, Rating, and Date of Rating
- ▶ Movie metadata available separately: Movie ID, Release Year, and Title
- ▶ Dataset is highly sparse — each user has rated only a tiny fraction of available movies

## 1.2 Preprocessing Pipeline

To handle the scale of the dataset within available computational constraints, a carefully designed preprocessing pipeline was implemented in `preprocessing.py`. Rather than using the full 100M+ rating corpus, a memory-safe subset strategy was adopted with the following steps:

- ▶ Parsed raw Netflix .txt files (`combined_data_1.txt` through `combined_data_4.txt`) using a streaming reader that identifies movie sentinel lines and associated rating records
- ▶ Removed duplicate (user, movie) pairs by retaining only the most recent rating per pair
- ▶ Applied iterative co-filtering: users with fewer than 20 ratings and movies with fewer than 50 ratings were removed, iterated to convergence
- ▶ Capped the dataset at the 50,000 most active users and 5,000 most-rated movies to fit memory constraints
- ▶ Built contiguous 0-based index maps for users and movies (`user2idx`, `movie2idx`) required for sparse matrix construction
- ▶ Serialised the cleaned ratings to a Parquet cache for fast reloading

### Train / Validation / Test Split

A per-user temporal split strategy was used to prevent data leakage and to mimic real recommendation scenarios. For each user, ratings were sorted chronologically: the most recent 20% formed the test set, the next 10% formed the validation set, and the remaining 70% constituted the training set. Users with fewer than 5 ratings were placed entirely in the training split. This design ensures the model learns from the past and is evaluated on genuinely unseen future interactions.

## 2. Methodology

---

Two distinct recommendation approaches were designed and implemented from the ground up. Each is grounded in a different philosophical view of how user preferences are captured:

- ▶ Item-Based Collaborative Filtering (Item-CF) finds items that tend to be rated similarly, and predicts how a user would rate a new item based on their prior ratings of similar items.
- ▶ Matrix Factorisation via Singular Value Decomposition (SVD) learns a compressed representation of both users and items as latent factor vectors, where the dot product of user and item factors approximates the observed rating.

### 2.1 Item-Based Collaborative Filtering

Implemented in `collaborative_filtering.py`, the `ItemBasedCF` class provides memory-efficient, explainable recommendations grounded in item similarity. The algorithm proceeds in four stages:

#### Stage 1: User Mean Centering (Adjusted Cosine)

Standard cosine similarity computed on raw ratings is susceptible to individual rating bias — a user who consistently rates everything 2 stars lower than average will appear dissimilar to a user who rates the same movies 2 stars higher, even if their relative preferences are identical. Adjusted cosine corrects for this by subtracting each user's mean rating from all of their observed ratings before computing similarity, centering the vectors at zero. Unrated entries remain zero and are excluded from the mean computation.

## Stage 2: Batch Item–Item Similarity Computation

The mean-centred rating matrix (shape: users × movies) is transposed so that each row represents a movie's rating vector across all users. Cosine similarity is computed in batches of 500 items at a time to avoid RAM exhaustion on large matrices. Self-similarity is zeroed out, and for each item only the top-50 most similar neighbours are retained (top-K pruning), yielding a compact similarity index of shape (n\_movies, 50).

## Stage 3: Rating Prediction

To predict user *u*'s rating for movie *m*, the model retrieves movie *m*'s top-50 similar neighbours and intersects them with movies already rated by user *u*. The prediction is a similarity-weighted average of *u*'s ratings on those neighbours. Only neighbours with positive similarity scores contribute. If no intersection exists, the user's historical mean rating is returned as a fallback.

## Stage 4: Recommendation Generation

For each user, all unseen movies are scored using the `fast_score` helper, which applies the same similarity-weighted logic. The top-N highest-scoring movies are returned as recommendations. The model additionally supports explainability: for any recommendation, it can generate a natural-language explanation of the form 'Because you liked [Movie A] and [Movie B], we recommend [Movie C]', citing the specific neighbour movies and the user's ratings for them.

Key implementation details: the similarity matrix is persisted to disk as a pickle file (`item_cf_similarity.pkl`) after the first computation, so subsequent runs load the cached matrix instantly rather than recomputing. The fit routine logs progress via `tqdm` at every batch and on completion.

## 2.2 Matrix Factorisation — SVD

Implemented in `svd_model.py`, the `SVDModel` class wraps the scikit-surprise library's SVD algorithm with a clean interface tailored to the Netflix Prize task. The objective function minimised during training is:

$$\min \sum (r_{ui} - \mu - b_u - b_i - p_u \cdot q_i)^2 + \lambda (\|p_u\|^2 + \|q_i\|^2)$$

where  $\mu$  is the global mean rating,  $b_u$  and  $b_i$  are learned user and item biases,  $p_u$  is the user latent factor vector (dimension 100), and  $q_i$  is the item latent factor vector (dimension 100). The regularisation term  $\lambda$  prevents overfitting to the sparse training data.

### Hyperparameters

- `n_factors = 100` (latent dimension — higher captures more nuance at greater compute cost)
- `n_epochs = 25` (SGD passes over all training ratings)
- `lr_all = 0.005` (learning rate for both factor vectors and bias terms)
- `reg_all = 0.1` (L2 regularisation weight applied uniformly)
- `biased = True` (user, item, and global bias terms included)

Predictions are generated by calling `algo.predict(user_id, movie_id)`, which computes  $\mu + b_u + b_i + p_u \cdot q_i$  and clips the result to the valid rating range [1, 5]. For Top-K recommendation generation, all unseen movies are scored and the highest-predicted ones returned. The `SVDModel` class also exposes hyperparameter tuning via `GridSearchCV` (optional), cross-validation, and direct access to learned latent factor vectors and bias terms for interpretability.

## 2.3 Model Hyperparameter Configuration

| Parameter            | Item-CF                   | SVD                       |
|----------------------|---------------------------|---------------------------|
| Similarity Measure   | Adjusted Cosine           | Cosine via SVD            |
| Neighbours / Factors | Top-50 similar items      | 100 latent factors        |
| Epochs / Passes      | Single pass (batch)       | 25 SGD epochs             |
| Learning Rate        | N/A                       | 0.005                     |
| Regularisation       | N/A                       | 0.1 (L2)                  |
| Min Common Raters    | 5                         | N/A                       |
| Bias Terms           | Implicit (mean centering) | Global + user + item bias |
| Training Library     | NumPy / SciPy / sklearn   | scikit-surprise           |

## 3. Evaluation Metrics

The evaluation pipeline, implemented in `evaluation.py`, covers two categories of metrics: rating-prediction accuracy metrics that measure how closely predicted scores match actual user ratings, and ranking quality metrics that assess the usefulness of the generated Top-K recommendation lists.

*Rank 1: Movie #3245 - Predicted Score: 4.82 | Rank 2: Movie #1897 - Predicted Score: 4.79 | ... (Top-10, unseen movies only)*

While SVD does not provide per-recommendation explanations out of the box, the `get_user_factors()` and `get_item_factors()` methods expose the learned latent vectors. These can be visualised using dimensionality reduction (e.g., PCA or t-SNE) to show user and movie clusters, or used to build post-hoc explanation systems.

## 6. Key Insights

### 6.1 The Rating Accuracy vs. Ranking Quality Trade-off

The most important finding of this study is the decoupling of rating accuracy and ranking quality. Item-CF achieves a lower RMSE — meaning its numerical rating predictions are more precise — yet SVD generates significantly better recommendation lists. This paradox is well-documented in the recommendation systems literature and has a clear explanation:

RMSE is computed over all test pairs uniformly, including low ratings. A model that accurately predicts a user's 2-star rating for a bad movie will score well on RMSE but this has no practical value for recommendation. Ranking metrics, by contrast, focus exclusively on what rises to the top: a model that surfaces a few genuinely loved movies (rating  $\geq 3.5$ ) in the top-10 achieves high MAP@10 even if its predictions for lower-rated movies

are less precise. SVD's latent factor representation is better suited to this task because it captures the global structure of taste — not just local item-item relationships.

## 6.2 Adjusted Cosine Effectively Controls for Rating Bias

The adjusted cosine similarity used in Item-CF demonstrably improves rating prediction over raw cosine. By centering each user's ratings at their personal mean, the similarity computation is freed from individual rating scale differences. A user who rates everything 1–2 stars lower than average (a 'harsh rater') will correctly appear similar to a lenient rater who liked the same movies, because the relative pattern of preferences is what matters. This preprocessing step is a critical component of the Item-CF model's accuracy advantage.

## 6.3 Computational Cost Implications for Production

Item-CF's 38× slower RMSE evaluation and 16× slower ranking evaluation compared to SVD have direct implications for system design. At serving time, SVD recommendations can be computed in milliseconds per user using pre-computed factor vectors. Item-CF requires accessing the similarity index and the user's rating history for each candidate movie — a process that scales with the number of candidates and the user's rating history length. For a production system serving millions of users, SVD (or its ALS/NMF variants) would be strongly preferred from an infrastructure standpoint.

## 6.4 Dataset Sparsity as the Dominant Challenge

The Netflix Prize Dataset's extreme sparsity — users have rated only a tiny fraction of available movies — is the single largest challenge for both models. Item-CF is most reliable when a target movie has many shared raters with the user's rated movies; when overlap is minimal, it falls back to the user mean. SVD handles sparsity more gracefully through its compressed latent space, but even its ranking metrics are limited by the low probability of any top-10 list coinciding with test preferences. Both models would benefit substantially from additional signals: implicit feedback (view duration, re-watches), content features (genre, director, cast), and temporal dynamics.

# 7. Future Improvements

---

## 7.1 Short-Term (High Impact, Low Effort)

- ▶ SVD Hyperparameter Tuning: Run GridSearchCV over `n_factors` (50, 100, 200), `n_epochs` (20, 40), `lr_all` (0.002–0.010), and `reg_all` (0.05–0.2). Expected improvement: RMSE reduction of 5–15%.
- ▶ Increase Training Data: Use all four Netflix data files (`combined_data_1–4.txt`) rather than only the first. Larger training sets consistently improve SVD performance.
- ▶ Item-CF Batch Prediction Optimisation: Vectorise the `predict_batch` loop in Item-CF to eliminate the Python-level for-loop bottleneck. NumPy CSR row slicing in batch mode can reduce RMSE evaluation time from ~340s to under 30s.
- ▶ Minimum Common Rater Threshold Tuning: Experiment with higher `min_common` values (10, 20) in Item-CF to filter out unreliable similarity scores from item pairs with few shared raters.

## 7.2 Medium-Term (Architectural Extensions)

- ▶ Alternating Least Squares (ALS): ALS is a parallelisable alternative to SGD for matrix factorisation, with better convergence properties on implicit feedback data. Libraries like implicit provide GPU-accelerated ALS implementations.
- ▶ Neural Collaborative Filtering (NCF): Replace the dot-product interaction with a multi-layer perceptron, allowing the model to learn non-linear user–item interactions. NCF typically outperforms SVD by 2–5% on ranking metrics.
- ▶ Temporal Dynamics: The Netflix dataset includes rating timestamps. Incorporating temporal decay (more recent ratings weighted more heavily) or temporal user bias (users change their taste over time) can significantly improve predictions for long-tenure users.
- ▶ Hybrid Systems: Combine SVD's latent factors with Item-CF's similarity scores as features in a learning-to-rank model (e.g., LightGBM or XGBoost). Hybrid systems consistently outperform individual models.

### 7.3 Long-Term (Research Directions)

- ▶ Graph Neural Networks for Recommendations (e.g., LightGCN): Model user–item interactions as a bipartite graph and use graph convolutions to propagate preference signals. State-of-the-art on multiple benchmarks.
- ▶ Large Language Model Integration: Use pre-trained LLMs to encode movie descriptions, reviews, and plot summaries into rich semantic embeddings, then use these as side-channel features alongside collaborative signals.
- ▶ Diversity and Serendipity Metrics: Optimise not only for accuracy but also for recommendation diversity (avoiding filter bubbles) and serendipity (surfacing genuinely surprising but relevant items).
- ▶ Cold-Start Solutions: Implement content-based fallback for new users (zero ratings) and new movies (zero ratings). A simple content-based filter using genre and release year can provide initial recommendations before collaborative signals accumulate.

## 8. Conclusion

---

This project successfully designed, implemented, and evaluated two recommendation system paradigms on the Netflix Prize Dataset. Item-Based Collaborative Filtering with Adjusted Cosine Similarity achieved superior rating prediction accuracy (RMSE 0.9422 vs. 1.0529), while Singular Value Decomposition delivered significantly better recommendation ranking quality across all four ranking metrics — MAP@10, Precision@10, Recall@10, and NDCG@10 — by factors of 2.2× to 2.4×.

The two models are complementary: Item-CF's strength lies in calibrated rating predictions and human-readable explanations, making it well-suited for scenarios where interpretability is paramount (e.g., audit trails, explanation-driven interfaces). SVD's strength lies in ranking quality and computational efficiency at serving time, making it the preferred backbone for large-scale production recommendation engines.

Both models demonstrate the fundamental challenge of extreme sparsity in user–item interaction data. The low absolute ranking metric values across both models reflect this inherent difficulty, not a limitation of the implementations. Future work should focus on incorporating implicit feedback signals, expanding the training corpus to the full Netflix dataset, and exploring hybrid architectures that combine the interpretability of collaborative filtering with the representational power of latent factor models.

---

*End of Technical Report*